

ST2AIM PROJECT · DEEP LEARNING

Teaching Computers to See

Cats vs. dogs image classification with a CNN

3,000 images · 1,043,905 parameters · 4 convolutional blocks · 4 models compared

Victor Haegeman — EFREI

1 The data

IMAGES

3,000

1,500 cats · 1,500 dogs

CLASS BALANCE

50 / 50

random baseline = 0.500

INPUT TENSOR

150 × 150 × 3

resized, rescaled to [0,1]

Set	Source	Images	Batches / epoch	Purpose
Training	train/ — 80%	1,600	16	fit the weights
Validation	validation/	1,000	10	monitor each epoch
Test	train/ — 20%	400	4	final score, used once



Why this is hard

Free poses, occlusions, humans in frame, variable lighting. Nothing is centred or normalised, unlike MNIST.

2

Why a CNN, not a dense network

One 150×150 colour image

67,500

input values $\rightarrow$ one dense layer of 1,000 neurons = 67.5 M parameters

Local connectivity

each neuron sees a 3×3 window, not the whole image

Weight sharing

the same kernel slides everywhere $\rightarrow$ parameter count independent of image size

The four building blocks

Layer	What it does	Learnable weights
Conv2D	M kernels slide over the input, one feature map each	$M \times (K \times K \times C_{in} + 1)$
ReLU	$\max(0, z)$ — the non-linearity; without it the stack collapses to one linear map	none
MaxPooling2D	keeps the max of each 2×2 window $\rightarrow$ halves height and width	none
Flatten + Dense	flattens the volume and classifies	$n_{in} \times n_{out} + n_{out}$

3 Architecture and parameter count

Layer	Output	Params
Rescaling 1/255	$150 \times 150 \times 3$	0
Conv2D 32 @ 3×3 + ReLU	$148 \times 148 \times 32$	896
MaxPooling 2×2	$74 \times 74 \times 32$	0
Conv2D 64 @ 3×3 + ReLU	$72 \times 72 \times 64$	18,496
MaxPooling 2×2	$36 \times 36 \times 64$	0
Conv2D 128 @ 3×3 + ReLU	$34 \times 34 \times 128$	73,856
MaxPooling 2×2	$17 \times 17 \times 128$	0
Conv2D 128 @ 3×3 + ReLU	$15 \times 15 \times 128$	147,584
MaxPooling 2×2	$7 \times 7 \times 128$	0
Flatten	6,272	0
Dense 128 + ReLU	128	802,944
Dense 1 + sigmoid	1	129

$$\text{params} = M \times (K \times K \times C_{\text{in}} + 1)$$

$$\text{block 2: } 64 \times (3 \times 3 \times 32 + 1) = 18,496$$

M = filters, K = kernel size, +1 = one bias per filter

TOTAL PARAMETERS

1,043,905

computed by hand, matches model.summary() exactly

77% sit in one dense layer

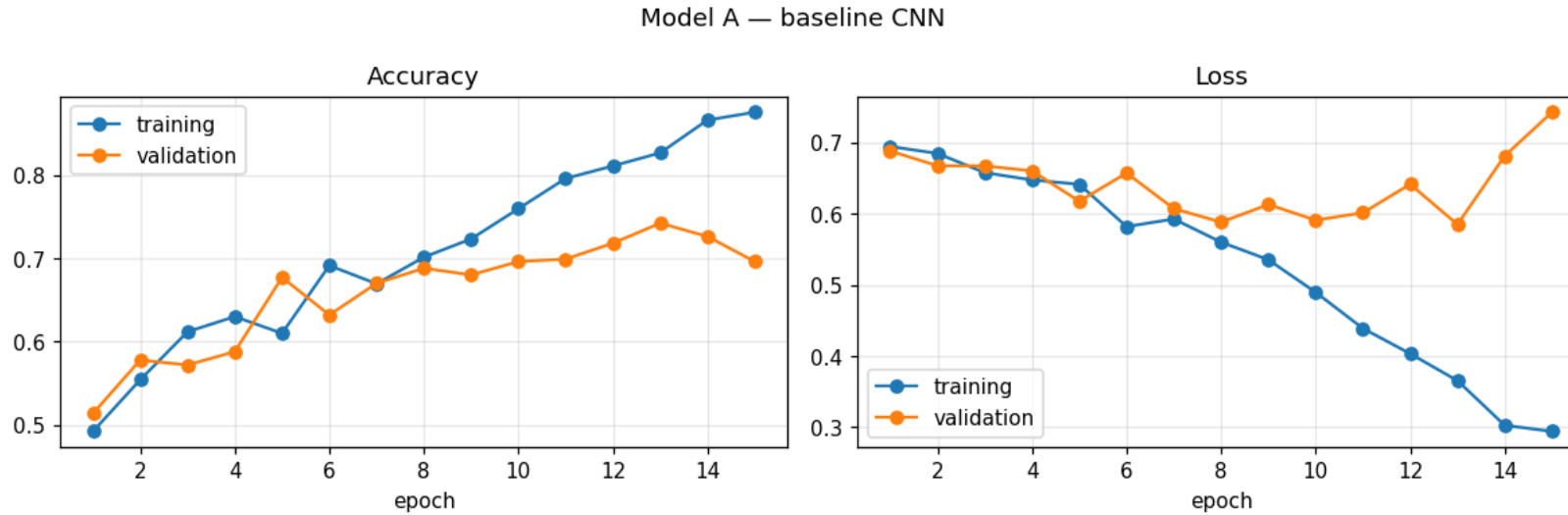
Dense(128) alone: 802,944 params.

All four conv blocks together: 240,832.

Weight sharing is what makes convolution cheap.

4

Training: the model memorises



AFTER 15 EPOCHS

0.875

training accuracy

0.696

validation accuracy

gap **0.179**

The evidence

Validation loss bottoms out at epoch 13

0.585

...then climbs back to

0.743

Training loss over the same window

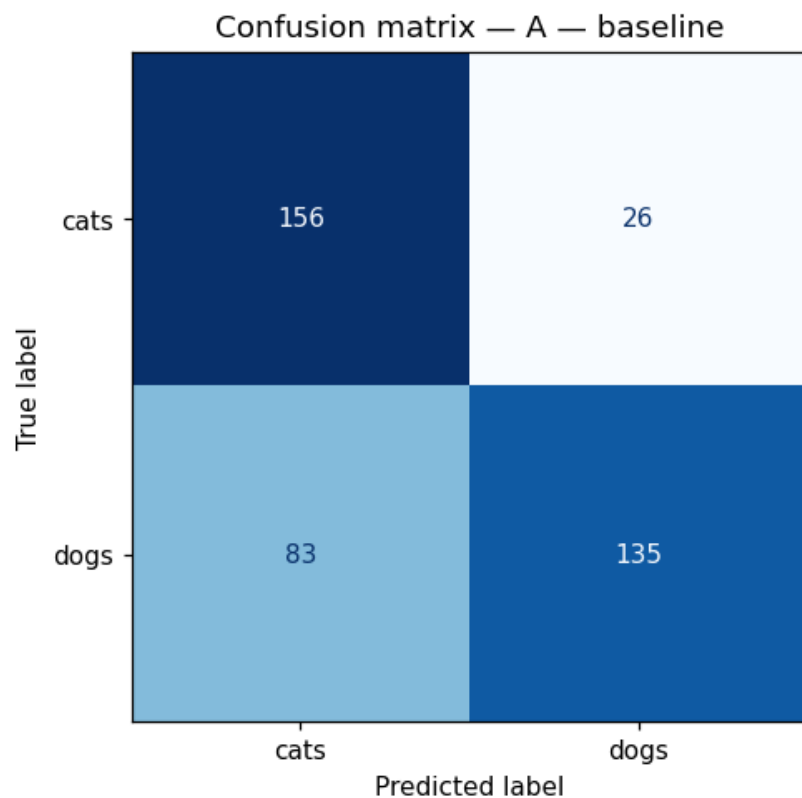
keeps falling

Three causes

- 1,600 training images only
- ≈ 650 parameters per image
- zero regularisation

5

Test set: 400 unseen images

**0.728****Accuracy**

above the 0.500 baseline — it learned real features

0.839**Precision**

when it says dog, it is right 84% of the time

0.619**Recall**

but it misses 38% of the real dogs

0.712**F1-score**

harmonic mean of the two

83 dogs classified as cats vs. 26 cats classified as dogs

The model is conservative on the dog class: at the slightest doubt it answers cat.
Accuracy alone would have hidden this.

6

Fix 1 — Data augmentation

Same image, eight random augmentations



Flip • rotation $\pm 10\%$ • zoom $\pm 10\%$, redrawn at every epoch

The network never sees the same image twice, so it cannot memorise.

Never applied to the test set

Keras augmentation layers are active in `fit()` and automatically inactive in `predict()`. The evaluation protocol is unchanged.

TRAIN / VAL GAP

0.179 → **0.009**

TEST ACCURACY

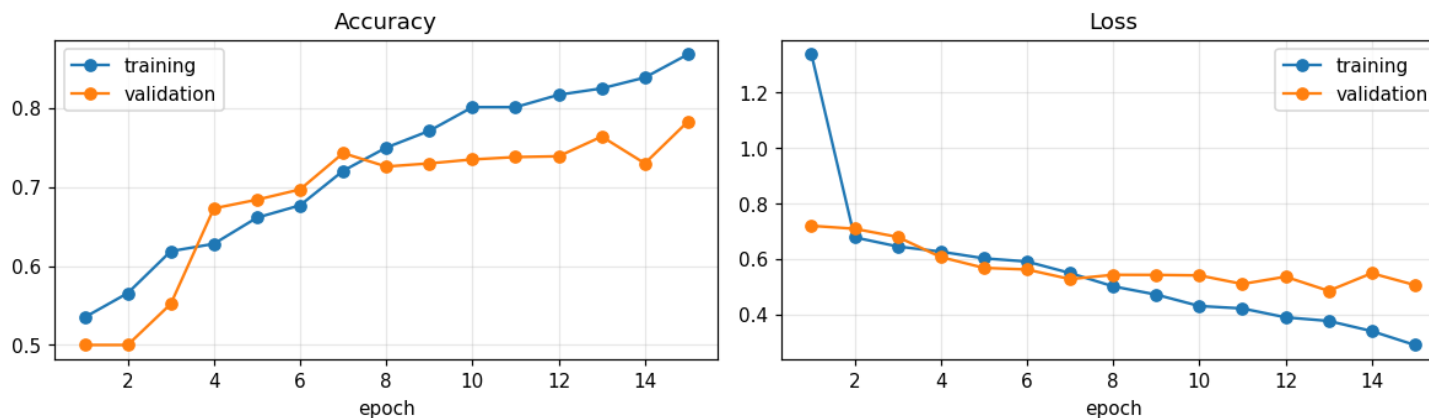
0.728 → **0.750**

RECALL ON DOGS

0.619 → **0.858**

7 Fix 2 — Dropout + batch normalization

Model C — dropout + batch normalization



TEST ACCURACY

0.780

best of the three · F1 = 0.810 · gap 0.085

Dropout 0.5

Half the dense units are switched off at each step → no unit can be relied on → distributed representation. Off at test time.

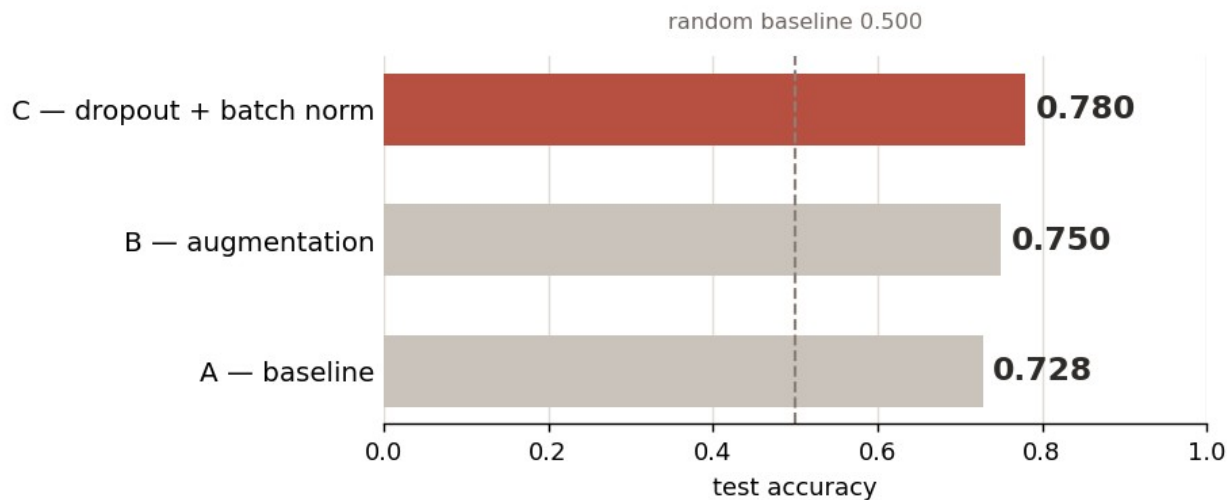
Batch normalization

Each convolution output is normalised over the batch → stable, faster training, mild regularisation.

Bug worth reporting: with the default momentum = 0.99 and only 240 updates, the running statistics never converge — validation accuracy stayed pinned at exactly 0.500 while training ran fine. momentum = 0.9 fixes it.

8 Three models, side by side

Model	Accuracy	Precision	Recall	F1	Train / val gap
A — baseline CNN	0.728	0.839	0.619	0.712	0.179
B — data augmentation	0.750	0.730	0.858	0.789	0.009
C — dropout + batch norm	0.780	0.766	0.858	0.810	0.085



Two levers, two profiles

Augmentation acts on the DATA and kills the gap. Dropout + batch norm act on the NETWORK and win on score.

Bonus — transfer learning

MobileNetV2 frozen, ImageNet weights, new head: ~1,300 trainable parameters instead of 1,043,905.

What the numbers say

1

The limit is the data, not the architecture

1,600 images, ~650 parameters per image. Both fixes improved generalisation without adding a single filter.

2

Regularisation does not add intelligence

it removes the option to memorise — that alone moved the gap from 0.179 to 0.009.

3

Read the curves, not just the final number

a single batch-norm hyperparameter pinned validation at 0.500 while training looked healthy.

Next: combine both · fine-tune MobileNetV2 · EarlyStopping · train on the full 25,000-image set

Thank you — questions?